

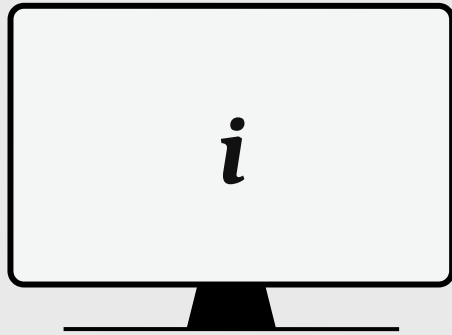
Programming Psychology Experiments: Course introduction & prerequisites

João Cardoso & Maxime Cauté

Session 1 | 23 September 2026

Disclaimer: these slides were created with Claude (AI), from the summary.md file in the GitHub repository, with additional prompting.

Today's plan



Course presentation

organisation + AI policy

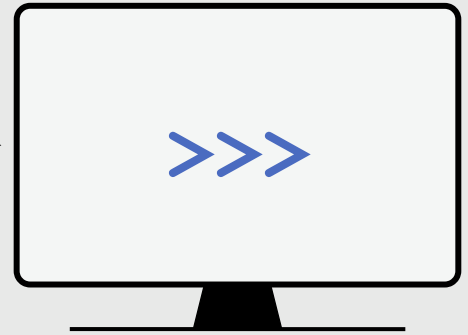
½ hour · M. & J.



Git & GitHub

version control

1 hour · M.



Expyriment warm-up

experimental code

1 hour · J.

Maxime Cauté



Research

Post-doc under Prof. S. Dehaene on the learning of mathematics (especially fractions!)

We conduct in-school experiments and interventions on small- and large-scale populations, as well as inside-the-lab experiments on adults (with finger-tracking)



PhD, December 2025

Procedural bugs when placing fractions on the number line and their conceptual sources: From diagnosis to remediation

Background

Former CogMaster student, with a linguistics major and a computer science background

The plan for the course

8 lectures of 2h30 · Wednesdays, 5.30–8 pm · may be adapted if needed

1	23/09	Course presentation · Git & GitHub · Experiment warm-up	<i>M. & J.</i>
2-3	30/09 · 07/10	Python (details TBA)	<i>J.</i>
4-5	14/10 · 21/10	Online experiments (details TBA)	<i>M.</i>
	28/10	<i>No class: autumn break</i>	
6	04/11	AI-assisted programming · Project setup	
		<i>pick a paper, discuss the design, implement the experiment</i>	
	11/11	<i>No class: bank holiday</i>	
7-8	18/11 · 25/11	Project presentations	

AI policy

LLMs are an increasingly useful tool for programming. You will eventually want to use them.

BUT

You won't learn if you use them all the time.

AND

AI has many shortcomings: it writes code that ends up needing an AI to manage it... but you want to know what's going on!

Understanding your code is your responsibility as a researcher — think of the replication crisis!

Today: please refrain from using AI — we're here to help!

We will come back to AI-assisted programming in session 6.

Git ≠ GitHub



Git

on your computer

- A software for (a) version control and (b) collaboration
- Runs locally — no internet needed



GitHub

on the web

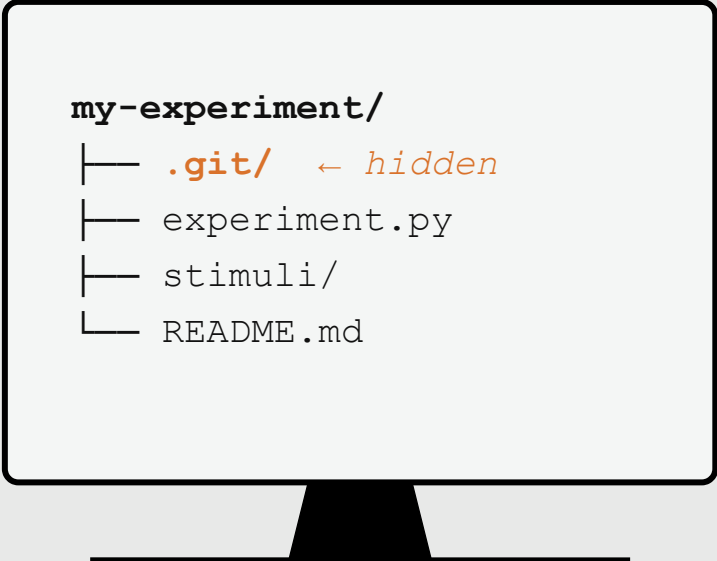
- A website/platform that hosts files using git
- Alternatives exist: GitLab, or host everything yourself!

Git: creating a repository

```
$ git init
```

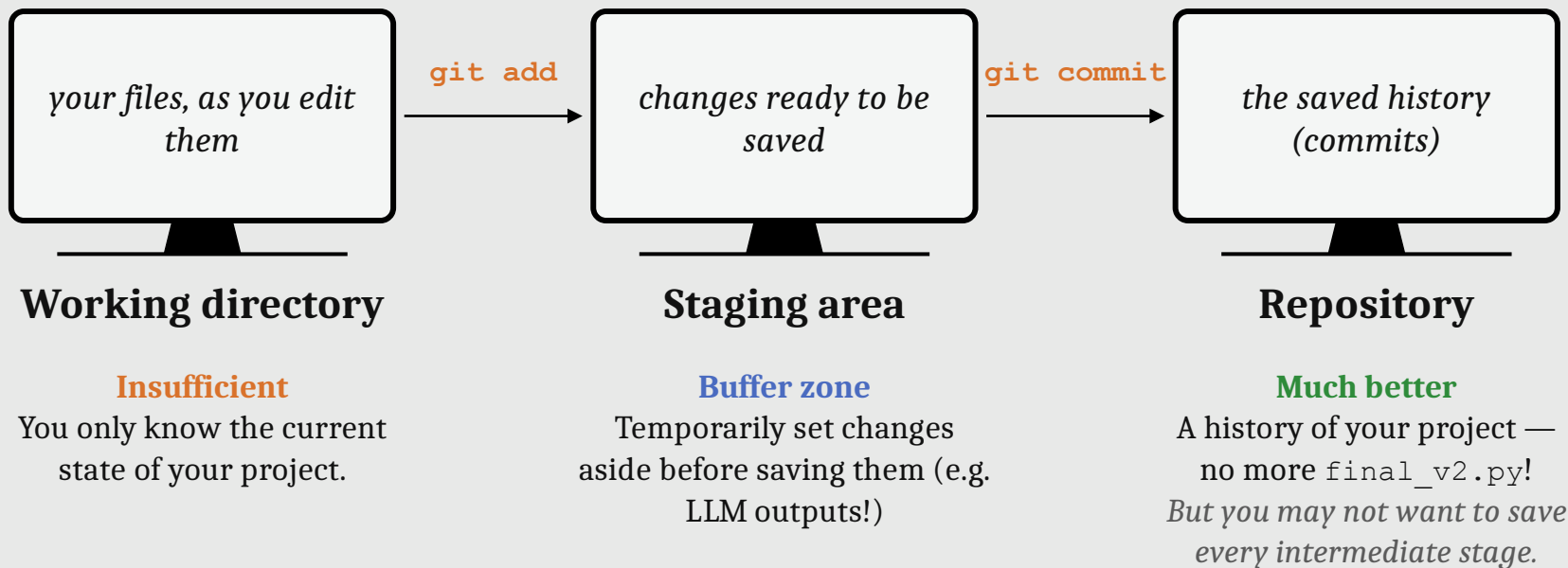
Turns a folder into a git repository.

Under the hood, it creates a hidden `.git` folder that contains everything of relevance: every saved version, every log message.



```
my-experiment/  
├── .git/ ← hidden  
├── experiment.py  
├── stimuli/  
└── README.md
```

Git: three stages



Git: the commands you need

COMMAND	WHAT IT DOES
<code>git status</code>	Shows the current state of your repository, especially what has changed
<code>git commit</code>	Creates a log entry (a saved version) of the current state
<code>git checkout</code>	Moves between versions (commits)
<i>Also very useful</i>	
<code>git add</code>	Stages modifications: a temporary save, made permanent by the next commit
<code>git diff</code>	Shows differences between working directory and staging area — great for reviewing what an LLM changed

Every commit comes with a message

```
git commit -m "your message here"
```

Vague

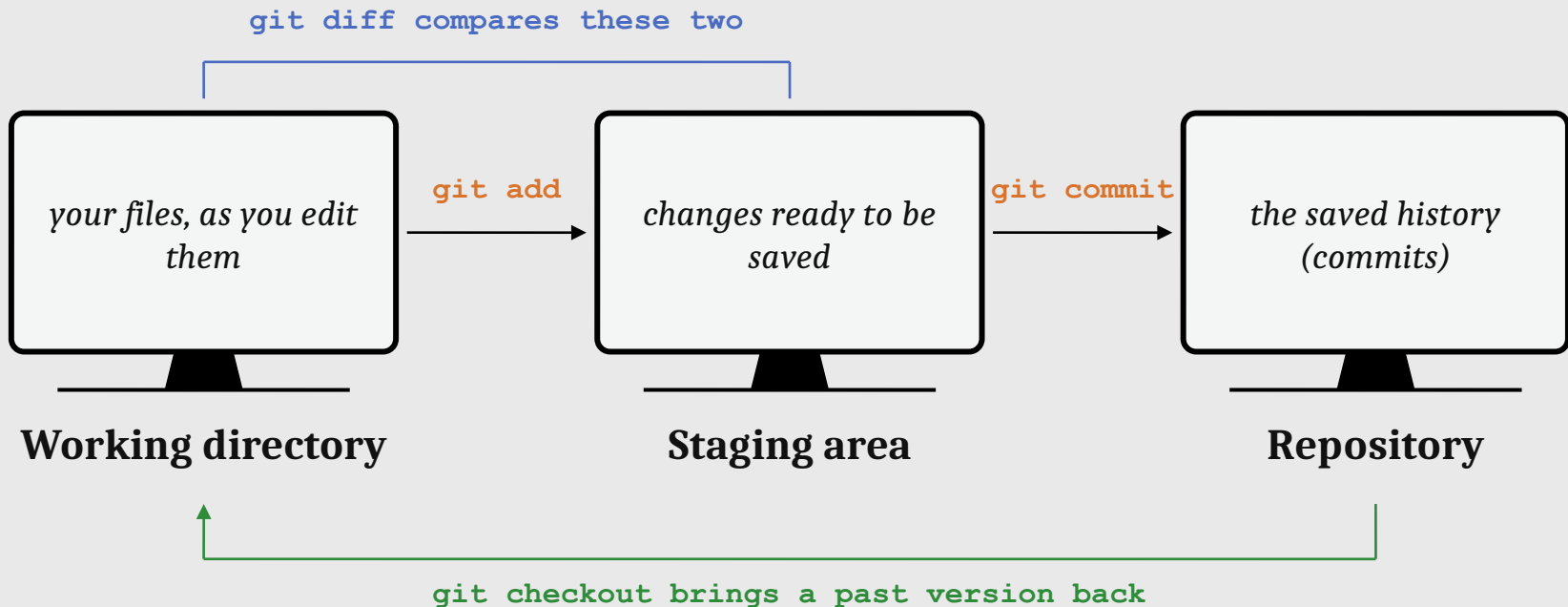
```
update  
fix  
changes  
final version 2
```

Precise

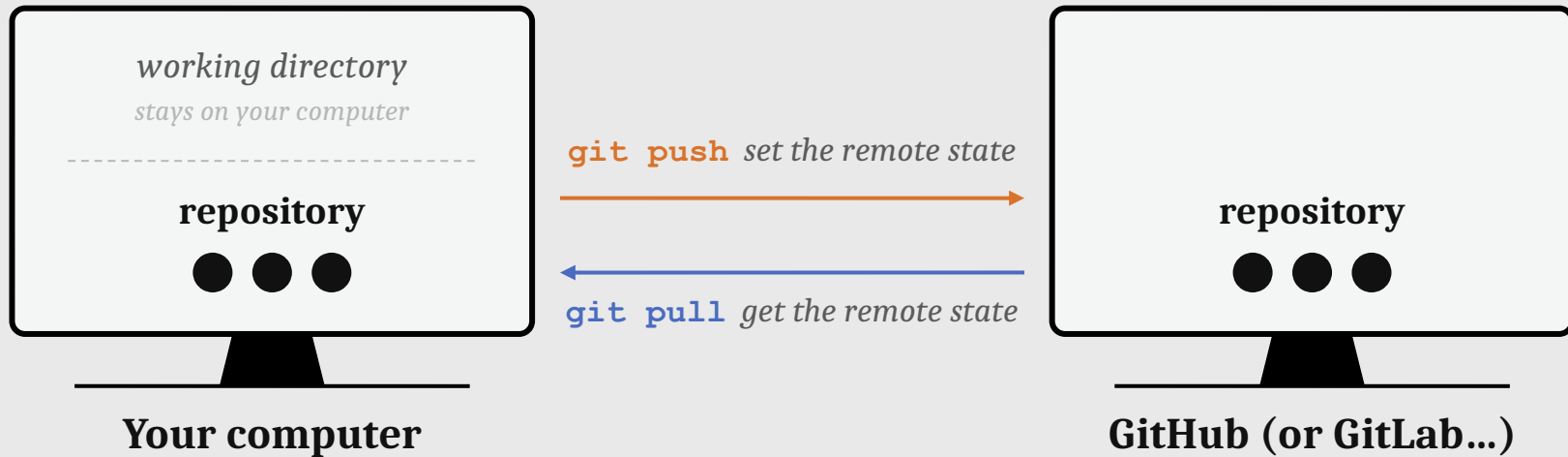
```
Add 500 ms fixation before trials  
Fix feedback after wrong key  
Save response times to CSV
```

Be as precise as possible: your future self (and your collaborators) will read these.

Git: three stages (additional commands)



Git: working with a remote



Only your commits are shared with the server! Uncommitted changes stay on your computer.

Hands-on 1: connect to GitHub

1

Create an account

on github.com (free)

2

Tell git who you are

use the same email as on GitHub

```
git config --global user.name "Your Name"  
git config --global user.email "you@email.com"
```

3

Generate an SSH key

press Enter to accept the defaults

```
ssh-keygen -t ed25519 -C "you@email.com"
```

4

Give GitHub your public key

Settings › SSH and GPG keys › New SSH key

```
cat ~/.ssh/id_ed25519.pub
```

5

Check the connection

expect: "Hi <username>! You've successfully..."

```
ssh -T git@github.com
```

On Windows, type these commands in Git Bash. Stuck? Raise your hand!

Hands-on 2: your first commit

1

Create a repository

on GitHub: New › name it ppe-sandbox › tick “Add a README file”

2

Clone it

SSH address: green “Code” button

```
git clone git@github.com:<you>/ppe-sandbox.git  
cd ppe-sandbox
```

3

Create a file, then look

e.g. notes.txt: it shows up as untracked

```
git status
```

4

Stage it

run git status again: it is now staged

```
git add notes.txt
```

5

Commit it

with a precise message!

```
git commit -m "Add notes on session 1"
```

Hands-on 3: the round trip

1

Push your commit

refresh the GitHub page: notes.txt is there

```
git push
```

2

Edit the README on GitHub

pencil icon › commit the change directly on the website

3

Pull it back

your local README now has the change

```
git pull
```

4

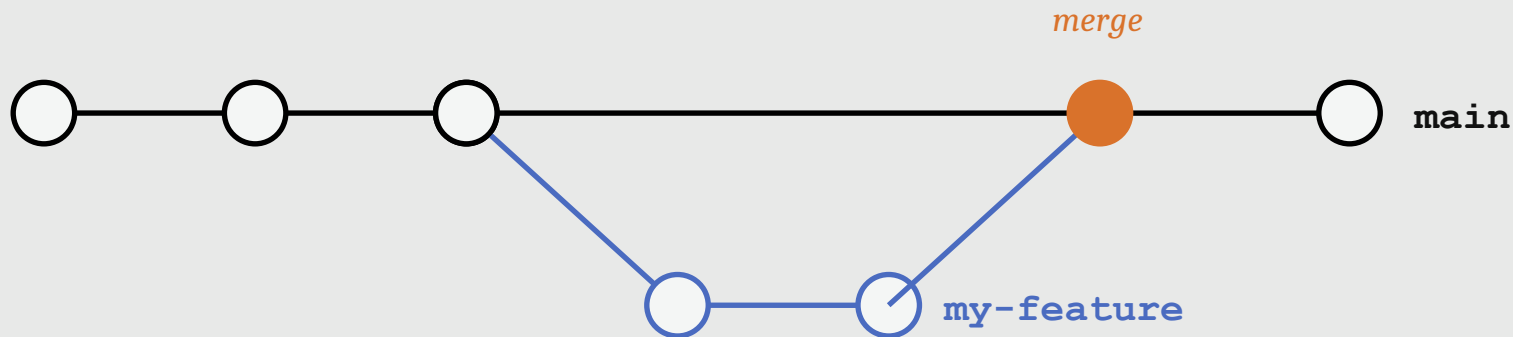
Look at the history

one line per commit, newest first

```
git log --oneline
```

Going further: travel back with `git checkout <commit-id>`, return with `git checkout main`

Bonus: branching and merging

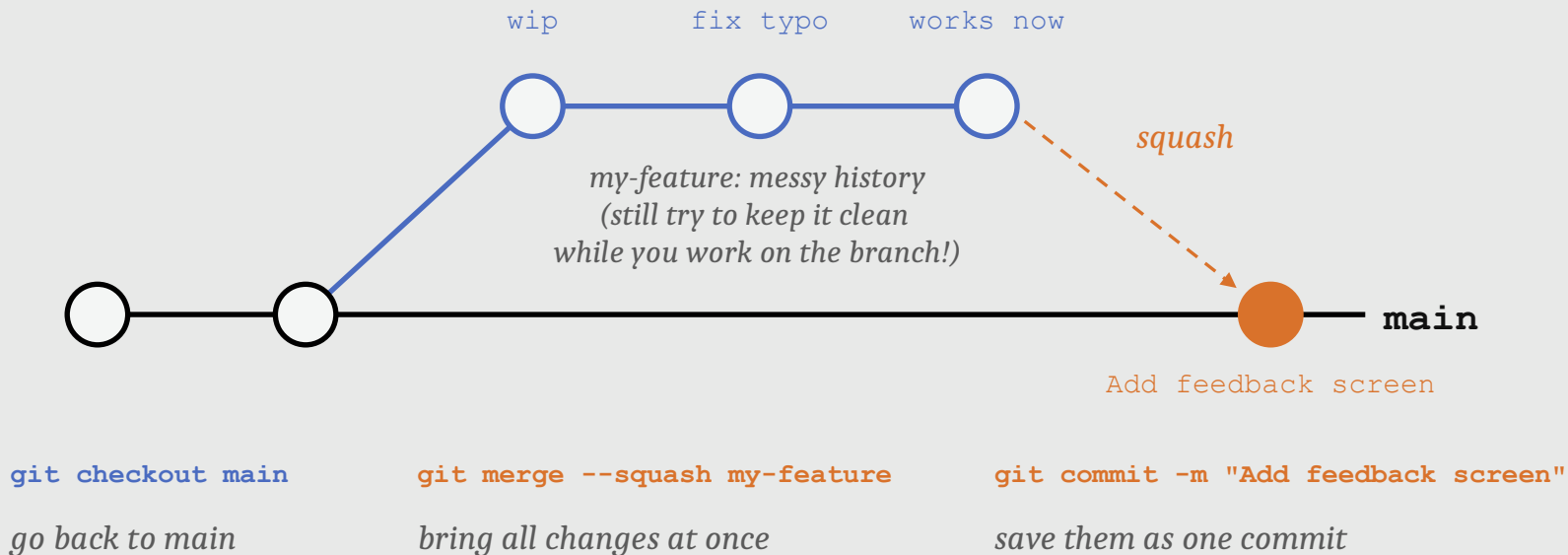


`git branch my-feature`
create a branch

`git checkout my-feature`
work on it (commit as usual)

`git checkout main`
`git merge my-feature`
bring it back into main

Bonus: squash merging



One clean commit per feature keeps the history readable (on GitHub: “Squash and merge”)